

LECTURE 26

Page Replacement Algorithms

CSC323 · Operating Systems

COMSATS University Islamabad, Lahore Campus

In This Lecture

- Why page replacement is necessary, and the role of the dirty (modify) bit
- **Reference strings**: the standard way to evaluate and compare replacement algorithms
- **FIFO**, worked in full, plus Belady's Anomaly — more frames causing more faults
- **Optimal (OPT/MIN)** replacement — provably best, and provably unusable in practice
- **LRU**, its implementation costs, and the Second-Chance (Clock) algorithm that approximates it cheaply
- **MFU/LFU** in brief, and thrashing — what happens when none of this is enough

Why Page Replacement Is Needed

Demand paging works beautifully right up until physical memory fills up. When a page fault occurs and no frame is free, the OS must choose an existing resident page, evict it, and reuse its frame for the newly faulted page.

- **Dirty (modify) bit** tracks whether a page was modified since it was loaded.
- If the evicted page was modified, it must be written back to disk before its frame can be reused.
- A page that was never written to can simply be discarded — an identical copy still exists on disk.

Every algorithm below answers one question: of all the pages currently resident, which one is safest to evict right now?

Reference Strings: A Common Basis for Comparison

A reference string is the sequence of page numbers a process references over time. Every algorithm in this lecture is evaluated against the same reference string, with the same number of frames, so the results can be compared directly.

The reference string used throughout this lecture

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2

13 page references, 3 frames available

FIFO Page Replacement

First-In-First-Out (FIFO) evicts whichever resident page has been in memory the longest, with no regard to how recently or how often it was actually used.

Reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2 · 3 frames

Ref	Frames (oldest → newest)	Fault?	Evicted
7	7, —, —	Fault	—
0	7, 0, —	Fault	—
1	7, 0, 1	Fault	—
2	0, 1, 2	Fault	7
0	0, 1, 2	—	—
3	1, 2, 3	Fault	0
0	2, 3, 0	Fault	1
4	3, 0, 4	Fault	2
2	0, 4, 2	Fault	3
3	4, 2, 3	Fault	0
0	2, 3, 0	Fault	4
3	2, 3, 0	—	—
2	2, 3, 0	—	—

Total FIFO faults: 10 out of 13 references.

Belady's Anomaly: More Memory Making Things Worse

Intuition says giving a process more frames should never hurt. FIFO is the counter-example: it is possible to construct a reference string where adding a frame increases the fault count.

Using the classic illustrative string 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 against FIFO

Frames	Page faults
3	9
4	10

Four frames — strictly more memory than three — produces one more fault, not fewer. This is a large part of why FIFO is rarely used alone in serious production systems, and why stack algorithms like LRU (mathematically guaranteed never to exhibit this anomaly) are generally preferred.

Optimal Page Replacement (OPT / MIN)

Optimal replacement evicts whichever resident page will not be used again for the longest time in the future. It is provably the best possible policy for a given reference string.

Reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2 · 3 frames

Ref	Frames After	Fault?	Evicted (reason)
7	7, —, —	Fault	—
0	7, 0, —	Fault	—
1	7, 0, 1	Fault	—
2	0, 1, 2	Fault	7 (never referenced again)
0	0, 1, 2	—	—
3	0, 2, 3	Fault	1 (never referenced again)
0	0, 2, 3	—	—
4	2, 3, 4	Fault	0 (next used at position 11 — farthest away)
2	2, 3, 4	—	—
3	2, 3, 4	—	—
0	2, 3, 0	Fault	4 (never referenced again)
3	2, 3, 0	—	—
2	2, 3, 0	—	—

Total OPT faults: 7 out of 13 references — the best any algorithm could do against this exact string.

LRU (Least Recently Used)

LRU evicts whichever resident page was least recently used — unused for the longest time in the past. It approximates OPT by betting the immediate past predicts the immediate future.

Reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2 · 3 frames

Ref	Frames (LRU → MRU)	Fault?	Evicted
7	7	Fault	—
0	7, 0	Fault	—
1	7, 0, 1	Fault	—
2	0, 1, 2	Fault	7
0	1, 2, 0	—	—
3	2, 0, 3	Fault	1
0	2, 3, 0	—	—
4	3, 0, 4	Fault	2
2	0, 4, 2	Fault	3
3	4, 2, 3	Fault	0
0	2, 3, 0	Fault	4
3	2, 0, 3	—	—
2	0, 3, 2	—	—

Total LRU faults: 9 out of 13 — one better than FIFO (10), one worse than OPT (7): $\text{OPT} \leq \text{LRU} \leq \text{FIFO}$.

Second-Chance (Clock) Algorithm

The **Second-Chance algorithm** (commonly called Clock) approximates LRU using only a single reference bit per frame. Frames are arranged in a circle with a clock hand pointing at the next eviction candidate.

- Whenever a page is referenced, its reference bit is set to 1 — at no extra cost beyond the reference itself.
- **On a fault, if the hand's frame has bit = 1:** give it a second chance — clear the bit to 0, advance the hand, don't evict.
- **On a fault, if the hand's frame has bit = 0:** evict it immediately, place the new page (bit = 1), advance the hand past it.

Clock Trace (Slot: Page:Bit)

Reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2 · 3 frames

Ref	Slot 0	Slot 1	Slot 2	Fault?	Evicted
7	7:1	—	—	Fault	—
0	7:1	0:1	—	Fault	—
1	7:1	0:1	1:1	Fault	—
2	2:1	0:0	1:0	Fault	7
0	2:1	0:1	1:0	—	—
3	2:1	0:0	3:1	Fault	1
0	2:1	0:1	3:1	—	—
4	4:1	0:0	3:0	Fault	2
2	4:1	2:1	3:0	Fault	0
3	4:1	2:1	3:1	—	—
0	4:0	2:0	0:1	Fault	3
3	3:1	2:0	0:1	Fault	4
2	3:1	2:1	0:1	—	—

Total Clock faults: 9 out of 13 — matching LRU exactly, at a fraction of the bookkeeping cost.

Fault-Count Comparison: $\text{OPT} \leq \text{LRU} \leq \text{FIFO}$

Same reference string (7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2), same 3 frames, every algorithm

7

OPT

9

LRU

9

Clock

10

FIFO

$\text{OPT} \leq \text{LRU} \leq \text{FIFO}$, exactly the ordering theory predicts. OPT (7 faults) is the unreachable lower bound; Clock (9) matches true LRU (9) at a fraction of the cost; FIFO (10), the simplest policy, pays for ignoring recency entirely.

MFU and LFU (Brief)

Two further counting-based policies occasionally appear, each tracking how often a page has been referenced rather than how recently.

- **LFU (Least Frequently Used)** evicts the page with the smallest reference count. Weakness: a page referenced heavily during an early burst keeps a high count forever, even long after the process stops needing it.
- **MFU (Most Frequently Used)** evicts the page with the largest reference count, reasoning that a heavily-used page is “done.” This rationale is weaker, and MFU is far less common in practice.

Both are mentioned mainly to round out the taxonomy — neither sees the widespread real-world use that LRU and its Clock approximation do.

Thrashing

Thrashing is the precise name for a process spending more time paging than executing, because its allocated frames are too few to hold its working set — the pages it is actively referencing right now.

The thrashing feedback loop



↻ CPU utilization drops further — the loop repeats, each lap worse than the last

The working-set model is the standard fix. Rather than admitting processes until memory runs out, it tracks each process's actively-referenced page set directly and only admits new processes when collective working sets still leave room.

Key Takeaways

- **Page replacement** is needed whenever a fault occurs with no free frame; a dirty (modify) bit determines whether the evicted page must be written back first.
- **Same reference string, 3 frames, every algorithm:** FIFO = 10, LRU = 9, Clock = 9, OPT = 7.
- **FIFO** can suffer Belady's Anomaly — more frames producing more faults (9 at 3 frames, 10 at 4, on the classic string).
- **OPT** is a theoretical lower bound; LRU approximates it from the past; Second-Chance (Clock) approximates LRU cheaply with one reference bit per frame.
- **Thrashing** — paging more than executing — can collapse system-wide CPU utilization; the working-set model is the standard defense.